

Capacitação: Construindo um Encurtador de URLs com Node.js e Express

1 Estrutura da Capacitação (1 hora)

1.1 Parte 1 — Introdução (5 min)

1.1.1 O problema

Pergunte:

Como o Bitly transforma uma URL enorme em algo pequeno?

Exemplo:

`https://www.youtube.com/watch?v=ABC123XYZ456`

vira

`https://bit.ly/abc123`

Explique que o sistema:

- Recebe uma URL;
- Gera um código;
- Salva a relação código → URL;
- Redireciona quando alguém acessa o código.

1.2 Parte 2 — O que é uma API REST? (5 min)

Mostre:

```
Cliente
  |
  v
Requisição HTTP
  |
  v
Servidor
  |
  v
Resposta HTTP
```

Exemplos:

GET /users

Buscar dados.

POST /users

Criar dados.

Explique rapidamente:

Método	Função
GET	Buscar
POST	Criar
PUT	Atualizar
DELETE	Remover

1.3 Parte 3 — Criando o Projeto (10 min)

1.3.1 Inicialização

```
npm init -y
```

1.3.2 Instalação

```
npm install express
```

Mostre o que apareceu:

- node_modules
- package.json
- package-lock.json

1.4 Parte 4 — Criando o Servidor (10 min)

Arquivo:

```
const express = require("express");
const app = express();

app.listen(3000, () => {
  console.log("Servidor rodando");
});
```

Explique:

```
express()
```

Cria a aplicação.

Porta:

```
app.listen(3000)
```

Pergunte:

O navegador conversa com quem?

Resposta:

```
localhost:3000
```

1.5 Parte 5 — Primeira Rota (5 min)

```
app.get("/", (req, res) => {  
  res.send("Olá mundo");  
});
```

Explique:

req Requisição.

res Resposta.

Teste no navegador:

```
http://localhost:3000
```

1.6 Parte 6 — Organizando Rotas (5 min)

Mostre o arquivo:

```
routes/Users.js
```

```
const router = express.Router();
```

Explique:

Antes:

```
app.get(...)  
app.post(...)  
app.get(...)
```

Tudo no mesmo arquivo.

Depois:

```
router.get(...)  
router.post(...)
```

Mais organizado.

1.7 Parte 7 — Criando o Encurtador (15 min)

1.7.1 Rota POST

```
router.post("/shorten", ...)
```

Recebe:

```
{  
  "url": "https://google.com"  
}
```

1.7.2 Middleware

```
app.use(express.json());
```

Explique:

Sem isso:

```
req.body
```

vem vazio.

1.7.3 Validação

```
if (!url)
```

Explique a importância de validar entradas.

1.7.4 Gerando o Código

```
console.log(  
  Math.random()  
    .toString(36)  
    .substring(2, 8)  
);
```

Saídas possíveis:

```
ab12cd  
xk92lm  
qwerty
```

1.7.5 Banco em Memória

```
const database = {};
```

Exemplo:

```
database["abc123"] = "https://google.com";
```

Resultado:

```
{  
  abc123: "https://google.com"  
}
```

1.7.6 Retorno

```
res.json({
  shortUrl: "http://localhost:3000/abc123"
});
```

Mostre no Postman ou Insomnia.

1.8 Parte 8 — Redirecionamento (5 min)

```
router.get("/:code")
```

Quando acessar:

localhost:3000/abc123

Busca:

```
database["abc123"]
```

Se encontrar:

```
res.redirect(originalUrl);
```

Explique o código HTTP 302.

2 Demonstração Final

2.1 1. Criar URL

POST /shorten

Body:

```
{
  "url": "https://www.google.com"
}
```

Resposta:

```
{
  "shortUrl": "http://localhost:3000/abc123"
}
```

2.2 2. Acessar

http://localhost:3000/abc123

Resultado:

Google